

Mining Software Process Lines

Fabian Rojas Blum^{*}
 Computer Science Department
 Universidad de Chile
 Santiago, Chile
 fblum@dcc.uchile.cl

ABSTRACT

There is a vast growth of generated event data being collected and stored by organizations. Within the field of Process Mining, this data has been used to discover, analyze and enhance processes from different domains. For this purpose there are hundreds of techniques available in different tools. These techniques are mostly focused on single processes. On the other hand, there are several proposals for dealing with multiple processes. Under different names, such as: configurable process models, process families or process lines, processes are characterized by capturing commonalities and variability between (similar) process models. These approaches have shown to be useful for organizations that have multiple variants of a given process, e.g., reducing redundancy and maintenance time. In this research proposal, we are developing a framework that allows the use of Process Mining techniques in families of processes within the software development domain (i.e., Software Process Lines).

Keywords

Software process lines, process mining, variability

1. INTRODUCTION

Small Software Enterprises (SSEs) define their software development processes in order to be able to guide, analyze, monitor, measure and improve them [9]. Moreover, a SSE may apply for a CMMI or ISO certification as long as they have a defined and used process. It gives them commercial advantage.

Software projects that are carried out in these companies can be quite varied, e.g., in size, complexity, team size, etc. For this reason, formalized processes must be tailored to each project before they can be used [2]. The set of all the possible software processes that a company follows, i.e., process variants, defines a process family. Enterprises can maintain a collection of predefined processes, picking

the variant that best applies to the project at hand; however, this approach does not scale well as it has co-evolution problems [1, 3]: processes are often redundant and different processes address similar problems. It also implies that an unnecessary effort being spent on unproductive tasks like process maintenance. In this sense, the Software Process Lines (SPrL) approach [8, 10], similar to Software Product Lines, proposes the formalization of this set of processes as a base process, which represents its common parts, along with its potential variability. Several existing notations can be used to formalize the SPrL [7]. Once it is defined, it can be configured, resolving variability and obtaining a specific process. In this way, SPrL promotes process asset reuse since common elements are defined only once, and specific processes are generated through process tailoring.

The way in which software is developed in companies is constantly changing and thus, the software development processes must evolve. Current techniques that support process improvement do not consider families of processes. An example of this is Process Mining (PM), that extracts knowledge from event data to support process improvement.

PM has been used for a variety of purposes such as understanding the reasons for bottlenecks, detecting and understanding deviations, predicting costs, risks and delays, recommending actions, supporting redesign and simulation. In order to do this, PM makes use of event logs, i.e., records related to the execution of a process. Event logs are records about significant events, like the beginning and ending of a software development activity. PM minimally requires that the events in the log can be chronological ordered, each event should be associated to both an execution trace (a particular sequence of activities that represent the execution of the process) and an activity. Different kinds of analysis can be applied depending on the available data. For example, activities performer might be used for social network analysis. The applications of PM fall under three main subfields: discovery, conformance and enhancement. Discovery takes an event log and produces a model without using any a priori information. In conformance, an event log is replayed on top of an existing process model to detect discrepancies and inconsistencies. During enhancement the model is extended or improved using information about the actual process recorded in an event log.

Even if no formal process is followed, SSE usually keep a register of the activities they carried out in their projects. The companies we work with keep project records in some format. These records can be converted into event logs in order to use PM techniques. However, PM techniques do not

^{*}Personal website: <http://users.dcc.uchile.cl/~fblum>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICSE '16 Companion, May 14-22, 2016, Austin, TX, USA

© 2016 ACM. ISBN 978-1-4503-4205-6/16/05...\$15.00

DOI: <http://dx.doi.org/10.1145/2889160.2889267>

consider SPrLs. In this way, companies using this approach and others that could benefit from it are not able to use PM for the improvement of their SPrLs. For this reason, we propose a framework that allows to apply PM similar techniques to SPrL. We will evaluate our approach within the 5 Chilean SSE we are currently working with.

Given this introduction as background for further discussion, the rest of this proposal is organized as follows: Section 2 states the problem. Section 3 discusses related work. Finally, section 4 describes the core of this ongoing work and is subdivided into four subsections, where we discuss the approach, evaluation, contribution and work progress.

2. PROBLEM STATEMENT

Even though the definition of a SPrL can benefit SSEs, they may not use this approach for several reasons. To start with, the definition of a SPrL is a time-consuming and error-prone task. Different employees have to meet over several sessions to discuss and specify the base process and its variability. We are currently working with 5 Chilean SSEs, and it took them 7-10 sessions to define a first version of their SPrL. Even when they are able to define a SPrL, it may not represent the process actually carried out by employees, as this kind of specification is biased by people's perception of how they think tasks should be carried out. We also found that during the execution of their projects, SSEs record process-related data that they generally do not use. These records correspond to different ways in which processes are executed (an implicit SPrL). This data could be used for process improvement. However, most current process improvement techniques do not consider SPrLs.

By applying PM techniques to SPrLs, we would allow companies to:

- discover the underlying SPrL in the case of enterprises that have not yet formalized their processes.
- in the case of enterprises that have defined a process, we may discover undocumented variability.
- improve based on real data, the SPrL of enterprises that have already defined it.

3. RELATED WORK

As far as we know, there are no proposals that process mining with the approach of software process lines from end to end. However, we found some work that is related to discovering families of processes.

Scramm et al. [13] present an empirical study where process variability in the form of variability operations applied in process tailoring is extracted from a series of process variants that have been applied along two years. However, their approach requires processes to be specified as models that conform to a specific and known metamodel. This is not the case in the small enterprises which we have been working with, so an alternative approach has been followed in order to deal with the kinds of available logs.

Weidlich et al. [14] present an approach for managing process variability. They provide a set algebra for behavioral profiles, i.e., abstractions for reasoning about order constraints. This is particularly useful for decoupled variants that may give different names to similar activities where similarities are identified mainly for their behavior. In the

case of software project logs, the data is stored altogether registering traces of activities of the same process, and therefore the situation is simpler.

The work of Buijs et al. [6] propose and compare four approaches for extracting configurable process models from event logs. A configurable process model is a representation of a business process, where activities can be *hidden* and paths may be *blocked* during configuration. The first two approaches merge individually discovered process models, whereas the second two approaches focus on discovering a single process model and its possible configurations (intuitively similar to our discovery approach). In all approaches, the result is a process model represented as a tree, where internal nodes document control flow and leaf nodes represent activities. Variability is modeled using node labels that guide tree traversal and a configured process model is created by exploring a tree branch. In the case of software project logs, the lack of separation between common and variable activities makes the models proposed in [6] harder to use for enactment, especially since looping behavior leads to node replications (as the output models are trees). Most importantly, the idea of alternative activities, a concept which is explicitly present in the software process domain (as there are different ways of carrying out a same activity), is not present in the authors' definition of configuration.

Santos et al. [12] present a way to discover variations in software development processes. It assumes there is an a priori known model and applies conformance checking with the log and this model. This conformance analysis generates a list of candidate "tailorable" process elements.

4. MINING SOFTWARE PROCESS LINES

In this section, we present our proposal for an end to end process mining for SPrL framework. Section 4.1 gives an overview of the proposed solution. Section 4.2 describes the proposed validation and evaluation steps. Section 4.3 discusses the expected contributions of this work. Finally, section 4.4 discusses the work that we have already carried out, including our publications related to this work. It also discusses ongoing and future work.

4.1 Approach

SSEs may or may not have a formalized process. Moreover, these enterprises usually store records related to the process they follow. These records may come in a variety of different formats including Excel sheets and issue tracking systems databases. Process related records can be transformed into event logs for further analysis.

We propose a SPrL Mining framework (see Figure 1). As in PM, this framework will consider all the phases (i.e., discovery, conformance and enhancement) with SPrLs. Shown in the center of the figure are the two input artifacts of our framework: event logs and the apriori known process models (optional). For this purpose, we plan to create, extend or improve traditional PM techniques so that they can deal with SPrLs.

Discovery (step (1) in figure 1) takes as input event logs and produces a SPrL that reflects the behavior seen in the log. It must be able to differentiate common from variable behavior, and representing these as a base process and variability, respectively. Conformance (step (2) in figure 1) replays the log on top of a model by checking if the behavior represented in the log correspond to possible executions in

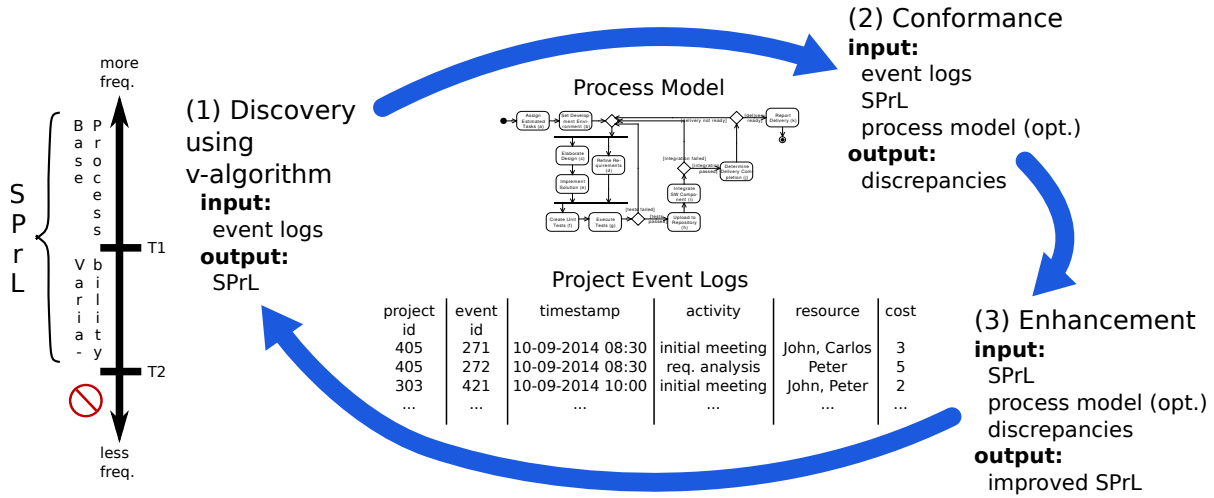


Figure 1: Software Process Line Mining Framework

the model in order to find discrepancies (i.e., behavior that is expressed in the log that cannot be replayed in the model). As we are working with SPrL, these discrepancies can also be used to update the variability model. Finally, enhancement (step (3) figure 1) builds an improved SPrL, taking as input the discovered SPrL, the discrepancies identified in step (2), the optional process model and other indicators (e.g., bottlenecks) that can be calculated depending on how much data there is in the log.

We have already built a prototype implementation of step (1), as an algorithm that we called the “v-algorithm” which is discussed in section 4.4.

4.2 Evaluation

As preliminary knowledge our research group found that there is a broad awareness of the importance of processes, although they are not as used in practice. This was discovered through a survey of Chilean SSEs where 61 out of the 139 contacted enterprises completed the survey. These and other data is publicly available ¹. We also know that enterprises record potentially useful data in different ways ranging from hand-made Excel sheets to databases from issue tracking systems (e.g., JIRA).

As event logs are the cornerstone for the application of mining techniques. In order to validate our approach we need to extract real data. For this reason, and following the methodologies from [11] and [15] we are currently applying a multiple descriptive case study in order to characterize the kinds of records that enterprises generate and can be potentially transformed into event logs. Within this case study we were able to extract event logs from 2 Chilean SSEs until now.

With these already collected event logs we were able to discover a SPrL with our algorithm and compare it manually with their already known process (conformance). We are going to build a prototype that will automate this and other (enhancement) tasks. Following the same methodologies as with our descriptive case study, we are going to develop case study protocols in order to evaluate each of

our developed techniques in terms of usability, usefulness and productivity. We will validate our findings by showing companies stakeholders the results of our techniques.

4.3 Contribution

This framework will improve the management of processes. It will be specially useful within enterprises where the diversity of their projects makes it advantageous to use the process lines approach and current PM techniques are not enough. Starting from process discovery, enterprises will be able to get a SPrL that accurately represents the way they work. In this way, we mitigate SPrL adoption problems (e.g., time-consuming task). Moreover, as process mining is based on factual data, the probability of capturing the real, as opposed to idealized, SPrL is improved.

Similar to current PM techniques, conformance will be able to replay logs on top of a priori known models which enables to make analysis of deviations. However, as we are considering SPrLs, other types of analysis have to be considered. Also, interesting questions arise. For example, should we consider a non-conforming path as a discrepancy or a variant of the process?

Finally, enhancement will allow the SPrL to be in continuous improvement. Again, this framework will have special considerations with respect to traditional PM techniques as it is based on families of processes. It will allow to see indicators for the whole SPrL, as well as specific variants. In this way it will allow SSEs to make informed decisions about whether they want to change the base process, its variability, or both.

4.4 Work Progress

Within our proposed framework we have mainly worked in discovery. Specifically, we have already proposed and published the v-algorithm [4, 5] (see left part of Figure 1). It was implemented as a PROM plug-in, a widely used academic tool for process mining. It is a SPrL discovery algorithm that, as other PM techniques, receives a log as input. The v-algorithm discovers the SPrL by dividing the behavior seen in the log in three based on two thresholds. The most common behavior is considered to build the base process. Less frequent but representative behavior is considered to build

¹<http://www.dcc.uchile.cl/gems/docs/InformeAnálisisEncuesta2015.pdf> (in spanish)

the variability model. Finally, the least frequent behavior is discarded. However this least representative behavior might be used for further analysis. For a complete description of the v-algorithm refer to [5].

We are now continuing the development of this framework, focusing on conformance and enhancement, as well as improving our v-algorithm so that it can discover more information about the models. For example, information about resources or work products can be used to refine the discovered SPRL (the current version of the algorithm only focuses on activity control flow). In parallel, as we want to evaluate our approach with real data, we will continue collecting records from SSEs.

Acknowledgments

The author would like to thank his advisors María Cecilia Bastarrica (cecilia@dcc.uchile.cl) and Jocelyn Simmonds (jsimmond@dcc.uchile.cl). This work is supported by CONICYT PhD Scholarship Program, Chile (CONICYT-PCHA/2015-21150841).

5. REFERENCES

- [1] O. Armbrust, M. Katahira, Y. Miyamoto, J. Münch, H. Nakao, and A. Ocampo. Scoping software process lines. *Software Process: Improvement and Practice*, 14(3):181–197, 2009.
- [2] O. Armbrust and D. Rombach. The right process for each context: objective evidence needed. In *Proceedings of the 2011 International Conference on Software and Systems Process*, pages 237–241. ACM, 2011.
- [3] M. C. Bastarrica, J. Simmonds, and L. Silvestre. Using megamodeling to improve industrial adoption of complex MDE solutions. In *Proceedings of the 6th International Workshop on Modeling in Software Engineering*, pages 31–36. ACM, 2014.
- [4] F. R. Blum, M. C. Bastarrica, and J. Simmonds. Software Process Line Discovery from (Noisy) Logs. In *Proceedings of REVASOFT’2014*, 2014.
- [5] F. R. Blum, J. Simmonds, and M. C. Bastarrica. Software process line discovery. In *Proceedings of the 2015 International Conference on Software and System Process*, pages 127–136. ACM, 2015.
- [6] J. C. Buijs, B. F. van Dongen, and W. M. van der Aalst. Mining configurable process models from collections of event logs. In *Business Process Management*, pages 33–48. Springer, 2013.
- [7] D. D. de Carvalho, L. F. Chagas, A. M. Lima, and C. A. L. Reis. Software process lines: A systematic literature review. In *Software Process Improvement and Capability Determination*, pages 118–130. Springer, 2014.
- [8] T. Martínez-Ruiz, F. García, M. Piattini, and J. Münch. Modelling software process variability: an empirical study. *Software, IET*, 5(2):172–187, 2011.
- [9] J. Münch, O. Armbrust, M. Kowalczyk, and M. Soto. *Software Process Definition and Management*. The Fraunhofer IESE Series on Software and Systems Engineering. Springer Berlin Heidelberg, Berlin, Heidelberg, 2012.
- [10] R. M. Pillat, T. C. Oliveira, P. S. Alencar, and D. D. Cowan. BPMNt: A BPMN extension for specifying software process tailoring. *Information and Software Technology*, 57:95–115, 2015.
- [11] P. Runeson, M. Host, A. Rainer, and B. Regnell. *Case study research in software engineering: Guidelines and examples*. John Wiley & Sons, 2012.
- [12] R. Santos, T. C. Oliveira, et al. Mining software development process variations. In *Proceedings of the 30th Annual ACM Symposium on Applied Computing*, pages 1657–1660. ACM, 2015.
- [13] J. Schramm, P. Dohrmann, and M. Kuhrmann. Development of flexible software process lines with variability operations: a longitudinal case study. In *Proceedings of the 19th International Conference on Evaluation and Assessment in Software Engineering*, page 13. ACM, 2015.
- [14] M. Weidlich, J. Mendling, and M. Weske. A foundational approach for managing process variability. In *Advanced Information Systems Engineering*, pages 267–282. Springer, 2011.
- [15] R. K. Yin. *Case study research: Design and methods*. Sage publications, 2013.